

ASSIGNMENT-8.1

2303A52429

VALLALA VYSHNAVI

BATCH-32

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

- Requirements:

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Example Assert Test Cases:

`assert is_strong_password("Abcd@123") == True`

`assert is_strong_password("abcd123") == False`

`assert is_strong_password("ABCD@1234") == True`

Expected Output #1:

- Password validation logic passing all AI-generated test cases.

```
def is_strong_password(password):
    if len(password) < 8:
        return False
    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    return has_upper and has_lower and has_digit

def main():
    password = input("Enter a password: ")
    if is_strong_password(password):
        print("The password is strong.")
    else:
        print("The password is weak. It should be at least 8 characters long and contain uppercase letters, lowercase letters, and digits.")

if __name__ == "__main__":
    main()

#testcase1
assert is_strong_password("Abcd@123") == True
#testcase2
assert is_strong_password("abcd123") == False
#testcase3
assert is_strong_password("ABCD@1234") == True
```

OUTPUT

The password is strong.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.
- Requirements:
 - o Classify numbers as Positive, Negative, or Zero.
 - o Handle invalid inputs like strings and None.
 - o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.
- Requirements:
 - o Ignore case, spaces, and punctuation.
 - o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

```

def classify_number(n):
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"

def main():
    number = float(input("Enter a number: "))
    result = classify_number(number)
    print(f"The number is {result}.")

if __name__ == "__main__":
    main()

#testcase1
assert classify_number(10) == "Positive"
#testcase2
assert classify_number(-5) == "Negative"
#testcase3
assert classify_number(0) == "Zero"

```

OUTPUT

```

Enter a number: 2429
The number is Positive.

```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.
- Requirements:
 - o Ignore case, spaces, and punctuation.
 - o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
```

```
assert is_anagram("hello", "world") == False
```

```
assert is_anagram("Dormitory", "Dirty Room") == True
```

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

```
def is_anagram(str1, str2):
    # Remove spaces and punctuation, and convert to lowercase
    str1 = ''.join(str1.split()).lower()
    str2 = ''.join(str2.split()).lower()
    return sorted(str1) == sorted(str2)

def main():
    word1 = input("Enter the first word: ")
    word2 = input("Enter the second word: ")
    if is_anagram(word1, word2):
        print(f"{word1} and {word2} are anagrams.")
    else:
        print(f"{word1} and {word2} are not anagrams.")

if __name__ == "__main__":
    main()

# AI-generated test cases
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

OUTPUT

```
Enter the first word: VYSH
Enter the second word: NAVI
VYSH and NAVI are not anagrams.
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

- Methods:

- o add_item(name, quantity)

o remove_item(name, quantity)

o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

```
assert inv.get_stock("Book") == 3
```

Expected Output #4:

- Fully functional class passing all assertions.

```
class Inventory:
    def __init__(self):
        self.stock = {}
    def add_item(self, name, quantity):
        if name in self.stock:
            self.stock[name] += quantity
        else:
            self.stock[name] = quantity
    def remove_item(self, name, quantity):
        if name in self.stock and self.stock[name] >= quantity:
            self.stock[name] -= quantity
        else:
            print(f"Not enough stock of {name} to remove.")
    def get_stock(self, name):
        return self.stock.get(name, 0)
print("All tests passed successfully.")
# AI-generated test cases
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
```

OUTPUT

```
All tests passed successfully.
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

- Requirements:

- o Validate "MM/DD/YYYY" format.
- o Handle invalid dates.
- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

```
def validate_and_format_date(date_str):
    from datetime import datetime
    try:
        date_obj = datetime.strptime(date_str, "%m/%d/%Y")
        return date_obj.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"

# AI-generated test cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
print("All tests passed successfully.")
```

OUTPUT

```
All tests passed successfully.
```